Autonomous Mobile Systems 2026/27

Home Assignment 1

This home assignment is about control of a wheeled mobile robot. The mechanical drive of the mobile robot is given in Figure 1. The configuration of the mobile robot is given as $\boldsymbol{q}^T(t) = [x(t),\ y(t),\ \varphi(t),\ \gamma(t)]$ and it is measurable, but the measurements are noisy. The robot inputs are the linear velocity of the front part $v_s(t)$ and the angular velocity of the front part $\omega_s(t) = \dot{\alpha}(t)$. The absolute velocities of the front wheels are limited to $v_{L,max} = v_{R,max} = 0.3\,\mathrm{m/s}$ and they have acceleration constraints $a_{L,max} = a_{R,max} = 1\,\mathrm{m/s^2}$.

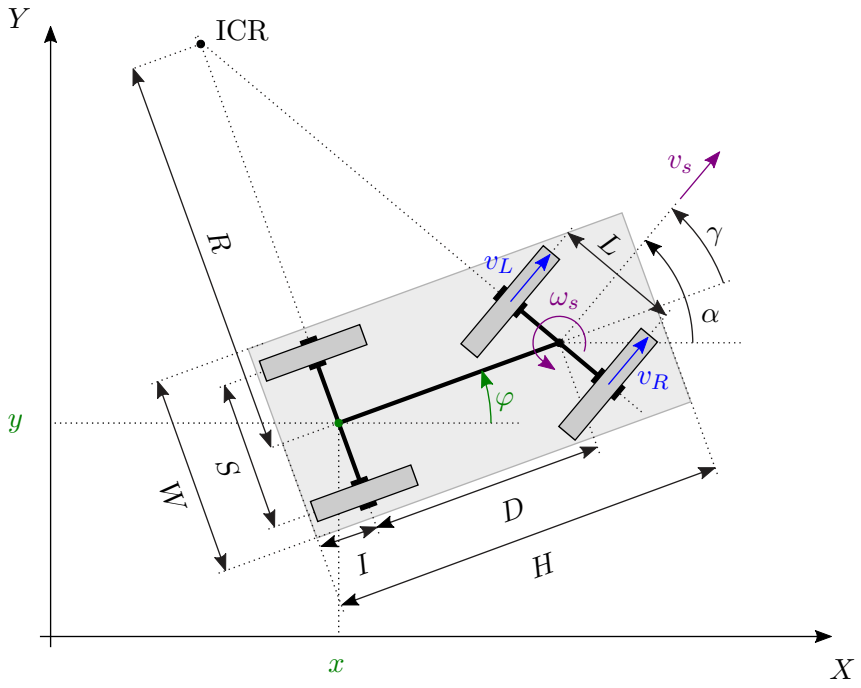


Figure 1: Structure of the mobile robot drive mechanism ($\omega_s(t) = \dot{\alpha}(t)$). Note that $\alpha = \varphi + \gamma$.

The distance between the front wheels is $L = 0.043\,\mathrm{m}$ and the distance between the rear axle and the center of the front wheels is $D = 0.1207\,\mathrm{m}$. The width of the mobile robot is $W = 0.096\,\mathrm{m}$ and its length is $H = 0.193\,\mathrm{m}$. The rear axle is $I = 0.023\,\mathrm{m}$ away from the rear end of the robot.

In this home assignment you will have to develop control algorithms for an environment that represents a parking lot, depicted in Figure 2. The parking lot contains eleven parking spots, marked as $\mathrm{P1}, \mathrm{P2}, \ldots, \mathrm{P11}$. Each parking spot also has a prescribed parking orientation. The margin around each parking spot is $G = 0.02\,\mathrm{m}$. The robot is considered to be inside the parking area when its entire outline lies within the parking spot enlarged by the margin G. The robots can drive in the free environment and also over parking areas. The robots cannot drive through obstacles and other robots. The light-blue area at the top is the starting area, where the robots are normally spawned. This home assignment is not about planning, but you can manually define one or more intermediate goals.

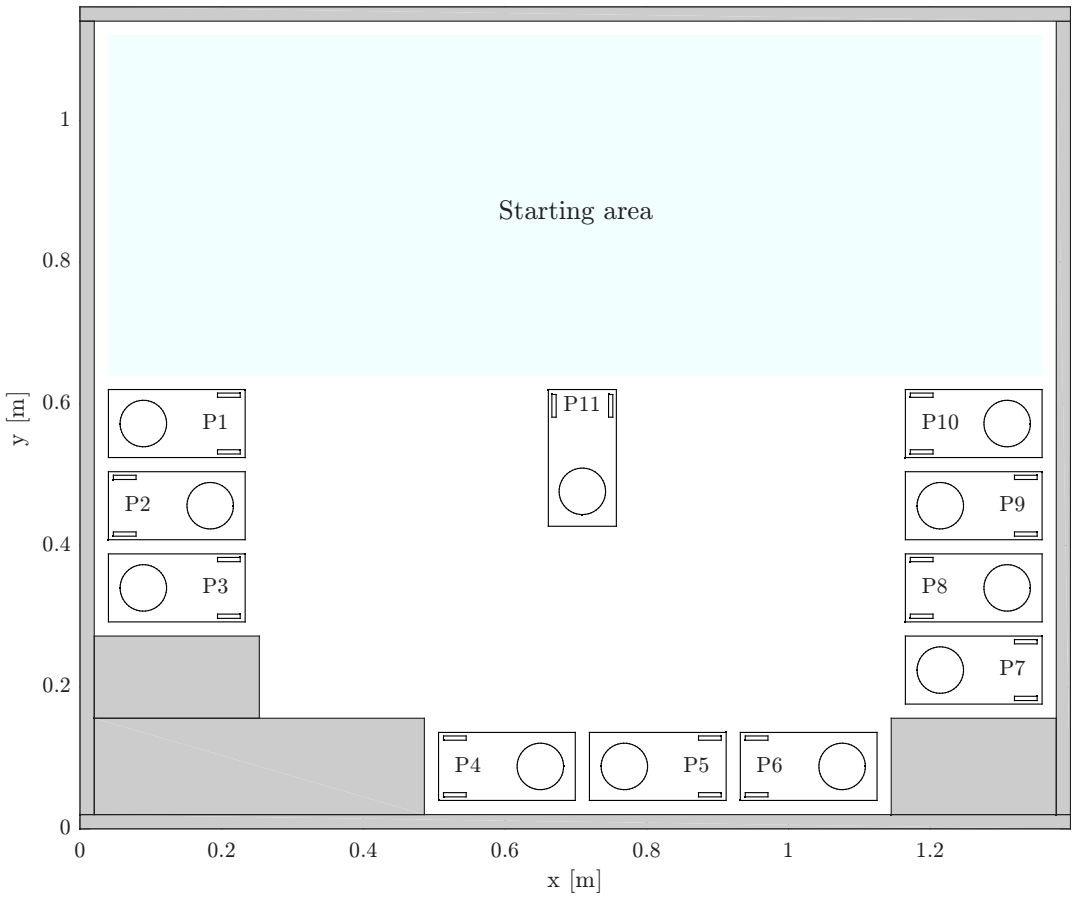


Figure 2: Environment with parking spots P1 to P11 (free space is white, free starting area is light blue and obstacles are gray)

Exercise 1

Write a control algorithm that drives the mobile robot from an initial configuration $\boldsymbol{q}_0^T$ to a goal pose $\boldsymbol{p}_{goal}^T = [x_{goal},\ y_{goal},\ \varphi_{goal}]$ (goal parking spot). The desired accuracy is 0.001 m for position and 0.5° for orientation. The control algorithm will be graded based on algorithm performance (i.e. final pose error, time, length and shape of the path, control effort, computational burden). The algorithm structure will also be graded.

Implement the control algorithm as a function in `controlParking.m`:

```
function [vs, ws, stop] = controlParking(q, goalPose, goalId,
    vehicleId, collision, inside, map)
```

In the function inputs you get the current configuration $\boldsymbol{q}$ (`q`) of the robot, the goal pose $\boldsymbol{p}_{goal}$ (`goalPose`) and the ID of the goal parking spot (`goalId`). The flag `collision` is raised when the robot collides with an obstacle or another robot, and the flag `inside` is set when the robot is in the goal parking area. The structure `map` contains all the information about the environment. Since only a single robot is used in this case, the `vehicleId` will be 1 — this parameter will be

useful later. The function must return velocity commands v_s and ω_s, and a stop signal should be emitted (i.e. set to `true`) when the goal is reached.

To test the control algorithm, use `simParking.m` that calls the function `controlParking` in a control loop. For example, to place the robot at the desired initial configuration $\boldsymbol{q}_0^T = [0.3, 0.9, 0, 0]$ and select the parking spot P11 as the goal location, run

```
simParking([0.3; 0.9; 0; 0], 11);
```

If the stop signal is not emitted, the simulation will end after 120 s. With the third argument in `simParking` you can disable noise if it is set to zero (disable noise only during development). With the fourth argument you can disable animation (if set to 0) or set animation decimation (1 for no decimation) to speed up the simulation.

Evaluate the performance for the following cases (although your solution should work for any case in the given environment):

- from $\boldsymbol{q}_0^T = \boldsymbol{q}_A^T = [0.3, 0.9, 0, 0]$ to P11,
- from $\boldsymbol{q}_0^T = \boldsymbol{q}_B^T = [0.9, 0.9, 0, 0]$ to P11,
- from $\boldsymbol{q}_0^T = \boldsymbol{q}_C^T = [0.4, 0.9, \pi, 0]$ to P11,
- from $\boldsymbol{q}_0^T = \boldsymbol{q}_D^T = [0.8, 0.7, \pi/2, 0]$ to P11,
- from $\boldsymbol{q}_0^T = \boldsymbol{q}_A^T$ to P1,
- from $\boldsymbol{q}_0^T = \boldsymbol{q}_A^T$ to P7, and
- from $\boldsymbol{q}_0^T = \boldsymbol{q}_A^T$ to P5.

You can also evaluate the performance of your solution for a randomly selected start configuration and goal parking spot:

```
simParking([], 0); % Selects random start configuration and goal
    parking spot
```

It is allowed to collide with obstacles, but try to minimize the number of collisions.

For testing purposes, you can manually drive the robot in the environment with your keyboard:

```
simManualDrive([0.3; 0.9; 0; 0], 11);
```

Note: You should run this command directly in the Matlab Command Window (manual driving does not work inside the Matlab Live Script). For the description of the keyboard commands, run

```
help manualDrive
```

Exercise 2

Also use `simParking.m` to test your control algorithm with multiple robots, driven one after another. This means that after the control algorithm emits a stop signal, the vehicle ID is increased (`vehicleId`) and your control algorithm should now drive the next robot that is spawned at some initial configuration $\boldsymbol{q}_0^T$ in the starting area to its goal. Goals are always unique, therefore robots have their goals in distinct parking spots. When a new robot is spawned, all internal states of the controller are reset (i.e. all persistent variables in the control function are reset).

Test your solution for the following cases (although your solution should work for any case in the given environment) when each robot is spawned at $\boldsymbol{q}_0^T = \boldsymbol{q}_S^T = [0.6, 0.9, 0, 0]$ and the sequence of parking spots is:

- P1, P8, P10, P3 and P6,
- P2, P9, P7 and P4,
- P1, P3, P10, P8, P6, P2, P7, P9, P4 and P5, and
- P11, P1, P2, P9, P10, P5, P3, P8, P7, P4 and P6.

You can run a simulation for five robots with a defined start configuration and goal parking spots:

```
simParking([0.6; 0.9; 0; 0], [1, 8, 10, 3, 6]);
```

or for five robots with random start configurations and random goal parking spots:

```
simParking([], [0, 0, 0, 0, 0]);
```

If needed, improve your solution so that it also works for multiple robots — this is the main task of this exercise.

For testing purposes, you can manually drive the robot in the environment with your keyboard (press keyboard key `g` to emit a stop signal and proceed to the next robot):

```
simManualDrive([0.6; 0.9; 0; 0], [1, 8, 10, 3, 6]);
```

Exercise 3

Upgrade the control algorithm from the previous exercises for a robot with limited rotation of the front part, $-\gamma_{max} \leq \gamma \leq \gamma_{max}$, where $\gamma_{max} = 30°$.

Implement the control algorithm as a new function `controlParkingLim.m`:

```
function [vs, ws, stop] = controlParkingLim(q, goalPose, goalId,
    vehicleId, collision, inside, map)
```

Evaluate the control algorithm for the same test cases as in the second exercise.

To run the simulation with the new control algorithm, run

```
simParkingLim([0.6; 0.9; 0; 0], [1, 8, 10, 3, 6]);
```

The function takes the same arguments as `simParking.m`.

For testing purposes, you can manually drive the robot in the environment with your keyboard (press keyboard key `g` to emit a stop signal and proceed to the next robot):

```
simManualDrive([0.6; 0.9; 0; 0], [1, 8, 10, 3, 6], 30/180*pi);
```

Remember the note from above that you should run this in the Matlab Command Window.

Submission

Provide the m-files with the control algorithms that you have implemented (`controlParking.m` and `controlParkingLim.m`) — these files should work with the unmodified versions of all the other

provided files for the simulation. The solutions should be minimalistic, methodical and explainable. Write a report as a *Matlab Live Script* named `homework.mlx` (use the provided template) that contains all your results, figures, additional source code if any, and also brief explanations, comments and observations. The code and report should be clear, concise, appropriately documented, and free of any unnecessary bloat and duplication. The results should be structured into sections and subsections: each section should represent one exercise and subsections should correspond to the exercise tasks/cases. All the required figures should also be drawn inside the *Matlab Live Script*. The source code should be executable without errors. The *Matlab Live Script* should be saved in the evaluated state (all the source code needs to be evaluated before the file is saved), so it can be reviewed even without running the source code. Follow the suggested naming conventions. Supplementary files can be provided only if really needed (include or reference them with links in the *Matlab Live Script*). Files that are not referenced in `homework.mlx` may not be considered in grading. The submitted solution must be the result of your own individual work. The use of any artificial intelligence tools must be clearly stated. An oral defense may be organized to verify the authenticity of the work and evaluate your comprehension and self-study. Submit the files before the deadline to the eFE e-classroom as a single ZIP-file (with a flat file structure without any folders). The ZIP-file should also include the MAT-files generated by the `homework.mlx`. You can create `homework.zip` from all files in the current directory with the command `prepare.m`.

Submission deadline: **12. 11. 2026**.